Data-Driven Surrogate Modeling for Lid-Driven Cavity Flows

Week 4 Lecture Notes

M&I-ENG 690A – AI in Fluid Mechanics

Summer 2026

Central question. How do we transform qualified CFD fields into fast predictive models without confusing interpolation, data leakage, or a visually plausible field with genuine physical validity?

Instructor: Ehsan Roohi

Contents

1 Where Week 4 Fits in the Course **2**

2 The Physical Dataset: Continuum Lid-Driven Cavity **2**
2.1 Why data qualification precedes machine learning 2

3 Three Learning Problems **3**
3.1 Parameter-to-scalar surrogate 3
3.2 Coordinate neural field 3
3.3 Operator learning 3

4 Train, Validation, and Blind Test **3**
4.1 The most dangerous leakage pattern 4

5 Nondimensionalization and Normalization **4**

6 Baselines Before Neural Networks **4**
6.1 Nearest-case baseline 4
6.2 Linear interpolation 4
6.3 Polynomial regression 4

7 Shallow DNN Surrogate **5**

8 Coordinate DNN for Complete Fields **5**

9 DeepONet: From Functions to Functions **5**
9.1 Worked POD prerequisite: representation versus prediction 6
9.2 What we implement in Week 4 6
9.3 Audit the physical pairing before fitting 6

10 Comparing Coordinate DNN and DeepONet **7**

11 Metrics for Fields **7**
11.1 Physical validation 7

12 Interpolation, Extrapolation, and Uncertainty **7**

13 Connection to Week 5: From Data Loss to Physics Loss **8**

14 Recommended Week-4 Workflow **8**

15 Common Failure Modes **8**

16 Summary **9**

1 Where Week 4 Fits in the Course

Week 1 introduced Python, TensorFlow, and a finite-difference lid-driven-cavity solver. Week 2 developed data-driven thinking and neural-network foundations. Week 3 introduced kinetic descriptions, DSMC, particle sampling, and statistical noise. Week 4 now connects these ideas: a numerical solver produces data, and a surrogate learns a map from physical inputs to quantities or fields of interest.

Lab2 scope and stopping rule. The released coordinate baseline in Lab2 predicts u, v. Direct zero-mean pressure prediction is implemented in Lab3 and the project notebooks. Lab2's fixed architectures stop on training loss for numerical convergence; this is not validation-based early stopping. A model-selection study must reserve complete validation cases before point expansion and fit scalers only on selection-training cases. Public held-out cases seen in earlier lessons are regression tests, not new untouched student evidence.

The week deliberately remains *data-driven*. Weeks 5–6 offer boundary-weighted and divergence-regularized project variants. The dedicated cavity PINN study appears later in Week 13, after students understand supervised surrogates and physical residuals.

Learning checkpoint

By the end of Week 4 you should be able to:

1. distinguish a scalar surrogate, coordinate neural field, and operator-learning model;
2. build a leakage-free training/validation/test split by flow case;
3. compare interpolation, polynomial regression, and a neural network;
4. evaluate a predicted cavity field using numerical and physical metrics;
5. explain the branch–trunk structure of DeepONet and identify when the word "operator" is justified.

2 The Physical Dataset: Continuum Lid-Driven Cavity

Consider a square cavity of side length L with a lid moving at speed U. The remaining walls are stationary. For an incompressible Newtonian fluid,

$$\mathrm{Re} = \frac{UL}{\nu}. \tag{1}$$

The Week-1 solver returns the nondimensional fields

$$u(x,y;\mathrm{Re}), \quad v(x,y;\mathrm{Re}), \quad \psi(x,y;\mathrm{Re}), \quad \omega(x,y;\mathrm{Re}). \tag{2}$$

The class production grid is fixed at 65×65 so that fields can be stacked without interpolation. A separate grid-sensitivity study may use 97×97 or 129×129, but those files are not mixed into the production dataset.

2.1 Why data qualification precedes machine learning

An ML model approximates its labels, not an inaccessible exact solution. If a CFD field is unconverged or grid-dependent, the network can learn that numerical error with high apparent accuracy. Each field therefore needs a quality card containing:

- final iterative residual and convergence history;
- finite-value and stability checks;
- Ghia centerline errors when a benchmark is available;

- divergence, boundary behavior, vortex location, and vortex strength;
- solver settings and student/case identity.

Important

Iterative convergence is not grid convergence. A residual of 10^{-8} only says that the discrete iteration stopped changing; it does not prove that the discrete solution is close to the continuum solution.

3 Three Learning Problems

3.1 Parameter-to-scalar surrogate

The simplest target is a vector of physically interpretable observables,

$$\mathrm{Re} \longmapsto \boldsymbol{q}(\mathrm{Re}) = [\psi_{\min},\ x_v,\ y_v]\,. \tag{3}$$

Here (x_v, y_v) is the location of minimum streamfunction and $\psi_{\min}$ measures primary-vortex strength. This problem has few training samples because one CFD case provides one input–output pair.

3.2 Coordinate neural field

A pointwise model uses

$$(\mathrm{Re}, x, y) \longmapsto (u, v). \tag{4}$$

Each grid point becomes a training row, but the statistically meaningful unit remains the complete Reynolds-number case. Grid points from one case are correlated and must not be randomly split between training and test sets.

3.3 Operator learning

A mathematical operator maps a function to another function:

$$\mathcal{G} : a(\xi) \longmapsto s(\boldsymbol{x}). \tag{5}$$

Examples include mapping a boundary velocity profile $U_{lid}(x)$ to a velocity field, or mapping an initial condition to a time-dependent solution. Operator learning differs from ordinary regression because both input and output are functions.

4 Train, Validation, and Blind Test

For this class, the split is made by Reynolds number. The test cases are hidden from fitting, normalization, architecture selection, early stopping, and loss-weight selection.

Role	Reynolds numbers	Permitted use
Training	100, 150, 200, 225, 250, 300, 350, 400	Fit parameters and scalers
Blind test	175, 275, 375	Final frozen evaluation only

With a small dataset, training cases may be internally assessed using leave-one-case-out cross-validation. A validation case can rotate through the training set. The blind cases remain unopened until the design is frozen.

4.1 The most dangerous leakage pattern

Suppose every 65×65 field is flattened and all $46{,}475$ rows from eleven fields are randomly split. The network then sees points from every Reynolds number during training. A small test error measures spatial interpolation inside already-seen cases, not generalization to a new flow condition.

Self-check

If 80% of the grid points from $\mathrm{Re} = 275$ are in training and 20% are in test, is $\mathrm{Re} = 275$ a blind test? **No.** The physical case has leaked.

5 Nondimensionalization and Normalization

Use nondimensional variables before statistical scaling:

$$x^* = x/L, \qquad y^* = y/L, \qquad u^* = u/U, \qquad v^* = v/U. \tag{6}$$

Then compute scaler parameters using training cases only. For example,

$$\widetilde{\mathrm{Re}} = 2\frac{\mathrm{Re} - \mathrm{Re}_{\min}^{train}}{\mathrm{Re}_{\max}^{train} - \mathrm{Re}_{\min}^{train}} - 1. \tag{7}$$

Coordinates already lie in $[0, 1]$, but mapping them to $[-1, 1]$ can suit tanh activations. Never recompute normalization using blind-test values.

6 Baselines Before Neural Networks

6.1 Nearest-case baseline

Choose the training field at the nearest Reynolds number. This establishes a minimum performance level.

6.2 Linear interpolation

For $\mathrm{Re}_a < \mathrm{Re}_* < \mathrm{Re}_b$,

$$\boldsymbol{q}(\mathrm{Re}_*) = (1-w)\boldsymbol{q}(\mathrm{Re}_a) + w\boldsymbol{q}(\mathrm{Re}_b), \qquad w = \frac{\mathrm{Re}_* - \mathrm{Re}_a}{\mathrm{Re}_b - \mathrm{Re}_a}. \tag{8}$$

The same formula applies pointwise to complete fields. Because the cavity family is smooth and one-dimensional in parameter space, this is a strong baseline.

6.3 Polynomial regression

For a scalar target,

$$q(\mathrm{Re}) = a_0 + a_1\widetilde{\mathrm{Re}} + a_2\widetilde{\mathrm{Re}}^{,2}. \tag{9}$$

A high polynomial degree is dangerous with eight training cases. Complexity must be selected using training-only cross-validation.

Research-level interpretation

A DNN that fails to beat linear interpolation on blind cases has not demonstrated predictive value. Neural complexity is not itself a scientific contribution.

7 Shallow DNN Surrogate

A scalar DNN takes normalized Reynolds number as input and returns three observables. For layer ℓ,

$$\boldsymbol{h}^{(\ell+1)} = \sigma\left(W^{(\ell)}\boldsymbol{h}^{(\ell)} + \boldsymbol{b}^{(\ell)}\right). \tag{10}$$

With so few flow cases, a compact model is preferable: two or three hidden layers with 16–64 neurons are adequate for demonstration. Use early stopping and repeat several random seeds.

The supervised loss is

$$\mathcal{L}_{data} = \frac{1}{N}\sum_{i=1}^{N} \|\widehat{\boldsymbol{q}}_i - \boldsymbol{q}_i\|_2^2. \tag{11}$$

This model is *not physics-informed*. Its labels came from CFD, but its loss does not enforce a governing equation.

8 Coordinate DNN for Complete Fields

The coordinate model receives $[\widetilde{\mathrm{Re}}, x^*, y^*]$ and predicts $[u^*, v^*]$. It shares one network across all cases and coordinates. Benefits include continuous query locations and modest memory. Limitations include spectral bias, weak treatment of global topology, and possible boundary violations.

To prevent large interior zones from dominating the loss, a zonal form may be used:

$$\mathcal{L} = \lambda_{int}\mathcal{L}_{int} + \lambda_{wall}\mathcal{L}_{wall} + \lambda_{corner}\mathcal{L}_{corner}. \tag{12}$$

Weights must be chosen without consulting blind tests. Week 5 will show how divergence or PDE residuals can be added as physics penalties.

9 DeepONet: From Functions to Functions

DeepONet contains two subnetworks. The **branch network** encodes samples of an input function,

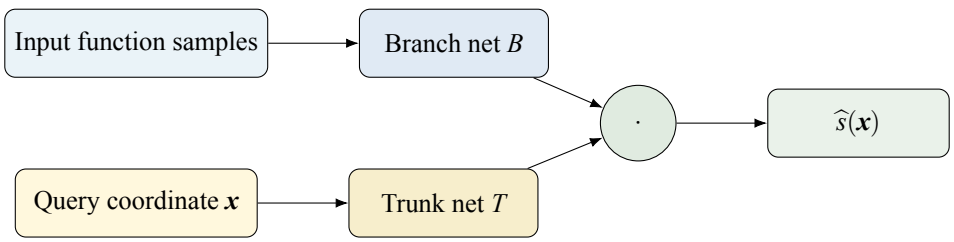
$$\boldsymbol{b} = B(a(\xi_1), \ldots, a(\xi_m)), \tag{13}$$

and the **trunk network** encodes an output coordinate,

$$\boldsymbol{t} = T(\boldsymbol{x}). \tag{14}$$

The scalar output is commonly written

$$\widehat{s}(\boldsymbol{x}) = \sum_{k=1}^{p} b_k t_k + b_0. \tag{15}$$



9.1 Worked POD prerequisite: representation versus prediction

Take centered snapshots

$$S = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix}, \qquad S^T S = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}.$$

The first singular value is $\sqrt{2}$, and one valid first mode is $\phi = (1,0)^T$. Its sign is arbitrary. For a query field $q = (2,.5)^T$, the exact coefficient is $a = \phi^T q = 2$ and the rank-one reconstruction is $(2,0)^T$. Its representation error norm is .5. A coefficient network that predicts 1.8 instead gives norm $\sqrt{.2^2 + .5^2} = \sqrt{.29}$. An exact regressor cannot recover a component omitted from the basis. **Exercise:** change the query to $(2,0)^T$. The representation error is zero, while the same predicted coefficient still gives error .2. Explain why cumulative training energy cannot alone establish physical fidelity at a new case. Classical Galerkin/DEIM dynamics are developed separately in the optional Week 4.1 notebook, with a hand-worked nonlinear sampling example.

9.2 What we implement in Week 4

The class notebook uses scalar Reynolds number in the velocity branch and $\log \mathrm{Re}$ in the pressure branch, with separate fixed POD spatial trunks:

$$\widehat{\boldsymbol{u}} = \overline{\boldsymbol{u}} + \sum_k b_k^u(\mathrm{Re})\, t_k^u(x,y), \qquad \widehat{p} = \overline{p} + \sum_k b_k^p(\log \mathrm{Re})\, t_k^p(x,y).$$

These spatial modes are computed from allowed CFD snapshots rather than learned by a coordinate network. This is a restricted parameter-to-field POD–DeepONet. It does not demonstrate learning arbitrary input functions. A function-input cavity dataset could vary the lid profile, with branch sensors $[U_{lid}(\xi_1), \ldots, U_{lid}(\xi_m)]$.

Important

Do not claim operator generalization from a dataset that varies only one scalar parameter. The architecture may be DeepONet-like, but the scientific claim is parameter-to-field generalization.

9.3 Audit the physical pairing before fitting

For case c and query j, a supervised pair is

$$(B_c, \boldsymbol{x}_{cj}) \longmapsto q_c(\boldsymbol{x}_{cj}).$$

Correct array shapes do not prove that these three entries belong together. Preserve source case IDs and source coordinates before flattening or shuffling. Compare branch case IDs to target case IDs and trunk queries to source target coordinates. In the cavity, check both x and y, meshgrid indexing, and the flattening order. A permutation can preserve every shape while destroying the pairing.

For a fixed-sensor branch, sensor positions and their order are shared across cases; the sampled function values vary. Query coordinates may vary across cases, provided each target retains its own query. Repeating the first case's coordinates is valid only after verifying that all query grids coincide.

The accompanying Lab 3 now includes an executable pre-fit audit and a negative control with deliberately permuted coordinates. Metadata copied from the same incorrect source cannot independently detect the error: preserve solver-export metadata and check its physical interpretation as well.

POD versus a learned trunk. The implemented POD–DeepONet uses a fixed spatial basis derived from allowed training fields and a scalar Reynolds-number branch. It is a restricted parameter-to-field model. A conventional function-input DeepONet instead encodes samples such as a lid-velocity profile and learns a coordinate trunk. Even a sampled geometry input from a low-dimensional family does not establish accuracy for arbitrary geometries.

Keep fitting and evaluation separate. During model selection, fit scalers, means, and POD bases on selection-training cases only. A declared final refit may include validation cases after choices are frozen; blind cases remain excluded. Split before expanding cases into rows. Different random seeds do not by themselves guarantee distinct physical cases. If inspected test results influence repairs or tuning, reserve a fresh final test. The Week 9 moving-throat nozzle audit applies these same rules to variable grids.

10 Comparing Coordinate DNN and DeepONet

The coordinate DNN mixes parameters and coordinates at its input. DeepONet keeps a separate representation of the case and the query location, then combines latent features. This separation can be beneficial when many functions and coordinates are available, but no architecture is guaranteed to win on a small dataset.

Property	Coordinate DNN	DeepONet-style model
Inputs	(Re, x, y) together	branch: Re; trunk: (x, y)
Output	(u, v)	latent inner products for (u, v)
Natural strength	simple neural field	separated case/coordinate encoding
Key risk	local fit without topology	overclaiming operator learning
Required baseline	field interpolation	field interpolation and coordinate DNN

11 Metrics for Fields

For a predicted velocity field,

$$E_{uv} = \frac{\sqrt{\sum_j [(\hat{u}_j - u_j)^2 + (\hat{v}_j - v_j)^2]}}{\sqrt{\sum_j [u_j^2 + v_j^2]}}. \tag{16}$$

Also report component MAE, maximum vector error, centerline error, wall-zone error, and inference time. Global metrics must be paired with physical checks.

11.1 Physical validation

$$E_{div} = \left[\frac{1}{N_{int}} \sum_{int} \left(\frac{\partial \hat{u}}{\partial x} + \frac{\partial \hat{v}}{\partial y} \right)^2 \right]^{1/2}, \tag{17}$$

$$E_{lid} = \frac{1}{N_{lid}} \sum_{lid} |\hat{u} - 1|, \tag{18}$$

$$E_{wall,n} = \frac{1}{N_{wall}} \sum_{wall} |\hat{\boldsymbol{u}} \cdot \boldsymbol{n}|. \tag{19}$$

Inspect centerlines, streamlines, vortex displacement, and corner zones. A low global error can coexist with a wrong secondary vortex or severe boundary error.

12 Interpolation, Extrapolation, and Uncertainty

Interpolation asks for a Reynolds number inside the training range and bracketed by training cases. Extrapolation asks for a condition outside the observed range. DNNs do not automatically respect scaling laws outside training data. A responsible report must label every test as interpolation or extrapolation.

An ensemble of independently initialized models provides a simple uncertainty indicator. Ensemble spread is not a rigorous confidence interval, but large disagreement can flag difficult regions. Calibration must be tested against actual errors.

13 Connection to Week 5: From Data Loss to Physics Loss

Week 4 trains

$$\mathcal{L} = \mathcal{L}_{data}. \tag{20}$$

The Week-5 project can add a boundary emphasis or divergence penalty. A fuller PINN objective, studied in the later dedicated module, is

$$\mathcal{L} = \lambda_d \mathcal{L}_{data} + \lambda_{bc} \mathcal{L}_{bc} + \lambda_{mom} \mathcal{L}_{momentum} + \lambda_{div} \mathcal{L}_{divergence}. \tag{21}$$

The transition is conceptually important: a supervised model imitates labels; a PINN or physics-regularized model additionally penalizes violation of known equations or constraints. Competing loss components can train at different rates, so loss weighting is a genuine optimization issue, not a cosmetic choice.

14 Recommended Week-4 Workflow

1. Run and qualify the common $\mathrm{Re} = 100$ CFD case.
2. Generate one assigned production field.
3. Merge only accepted cases and freeze the dataset.
4. Extract scalar vortex observables.
5. Fit interpolation and polynomial baselines.
6. Fit a compact scalar DNN.
7. Build a field-interpolation baseline.
8. Compare coordinate DNN and parameterized DeepONet-style models.
9. Freeze models and evaluate blind cases.
10. Report numerical errors, physical checks, and failures.

15 Common Failure Modes

Symptom	Likely cause	Required response
Very small test error	Grid-point leakage	Split by complete Reynolds-number case
DNN worse than interpolation	Insufficient data or overfitting	Report baseline honestly; simplify model
Good interior, poor walls	Loss dominated by interior points	Add wall metrics; consider zonal sampling
Wrong streamlines, modest L_2	Metric hides topology	Plot streamlines and vortex location

Excellent training, poor blind test	Memorization	Case-level cross-validation and smaller model
Unstable DeepONet result	Sparse function/case coverage	Reduce claim; repeat seeds; expand data

16 Summary

- Data quality is part of the ML method.
- The split must be by flow case, not by grid point.
- Interpolation is a serious baseline for a one-parameter cavity family.
- Numerical error and physical validity are different questions.
- A coordinate DNN learns a neural field; DeepONet separates case and coordinate representations.
- A scalar-parameter branch gives a DeepONet-style parametric surrogate, not full function-to-function operator learning.
- Week 5 adds physics constraints and PINNs to the same cavity problem.

References

1. U. Ghia, K. N. Ghia, and C. T. Shin, "High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method," *Journal of Computational Physics*, 48, 387–411, 1982.
2. L. Lu, P. Jin, and G. E. Karniadakis, "DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators," *Nature Machine Intelligence*, 3, 218–229, 2021.
3. M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378, 686–707, 2019.
4. S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," *Journal of Computational Physics*, 449, 110768, 2022.